สัปดาห์ที่ 7-8

Finite State Machine

Moore & Mealy

state diagram · state encoding · next-state logic · output logic · Moore FSM · Mealy FSM

Finite State Machine คืออะไร?

วงจรที่ "จำ" สถานะปัจจุบันไว้ และเปลี่ยนสถานะตาม input ตามกติกาที่กำหนดไว้ล่วงหน้า



ไฟจราจร

RED ⟶ GREEN ⟶ YELLOW ⟶ RED วนตามเวลา



ประตูกั้นทางเข้า

LOCKED / UNLOCKED ตามเหรียญที่หยอด



ลิฟต์

IDLE / MOVING_UP / MOVING_DOWN / DOOR_OPEN



ตู้ขายเครื่องดื่ม

รอเงิน ⟶ ครบราคา ⟶ จ่ายของ ⟶ ทอนเงิน

องค์ประกอบของ FSM

4 สิ่งที่ต้องนิยามให้ครบก่อนเริ่มเขียนโค้ดเสมอ

องค์ประกอบ	ความหมาย	ตัวอย่าง (ไฟจราจร)
State (สถานะ)	สถานะที่เป็นไปได้ทั้งหมดของระบบ ณ เวลาหนึ่ง	RED, GREEN, YELLOW
Input	สัญญาณจากภายนอกที่มีผลต่อการเปลี่ยนสถานะ	reset, enable, timer tick
Next-State Logic	กติกาว่า "สถานะถัดไป" คืออะไร จาก state ปัจจุบัน + input	ครบเวลา ⟶ ไปสถานะถัดไป
Output Logic	สัญญาณขาออกของระบบ	Moore: f(state) เท่านั้น · Mealy: f(state, input)

Moore vs Mealy

	Moore Machine	Mealy Machine
Output ขึ้นกับ	state ปัจจุบันเท่านั้น	state ปัจจุบัน + input ปัจจุบัน
จำนวน state	มักต้องใช้ state มากกว่า	มักใช้ state น้อยกว่า
Timing ของ output	เปลี่ยนพร้อม clock edge (sync)	เปลี่ยนทันทีที่ input เปลี่ยน (async)
Output glitch	น้อย (output ผูกกับ state)	เป็นไปได้ (input เปลี่ยนกลาง cycle)
เหมาะกับ	ระบบ sync ที่ output ต้องนิ่ง	ระบบที่ต้องตอบสนองเร็ว
ตัวอย่าง	Lab A: Traffic Light Controller	Lab B: Sequence Detector

ขั้นตอนออกแบบ Moore FSM

ทำตามลำดับนี้ทุกครั้ง จะไม่หลงทางเวลาโจทย์ซับซ้อนขึ้น

1. นิยาม State ทั้งหมด — ระบบมีสถานะอะไรบ้าง ตั้งชื่อให้สื่อความหมาย (RED, GREEN, YELLOW)
2. วาด State Diagram — วงกลมแทน state, ลูกศรแทนเงื่อนไขเปลี่ยนสถานะ
3. กำหนด State Encoding — แปลง state เป็นรหัสบิต (Binary หรือ One-hot)
4. เขียน Next-State Logic — combinational logic (case/if) จาก state ⟶ next_state
5. เขียน Output Logic — combinational logic จาก state ⟶ output เท่านั้น (Moore rule)

ประตูหมุนกั้นทางเข้า

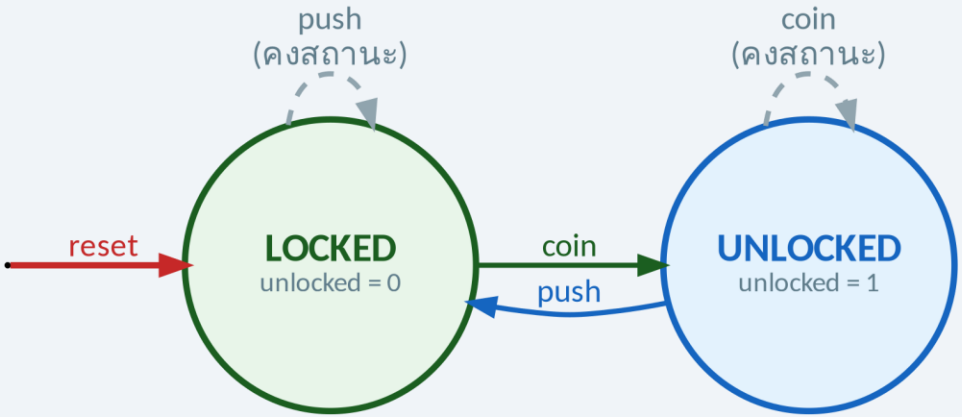
โจทย์คลาสสิกที่สุดสำหรับฝึก Moore FSM 2 สถานะ ก่อนไปเจอโจทย์จริงที่ซับซ้อนขึ้น

สเปก:

- 2 สถานะ: LOCKED, UNLOCKED
- Input: coin (หยอดเหรียญ), push (ดันประตู)
- Output: unlocked (ปลดล็อกประตู)

กติกา:

- LOCKED + coin ⟶ UNLOCKED
- UNLOCKED + push ⟶ LOCKED
- กรณีอื่น ⟶ คงสถานะเดิม (self-loop)



ตารางสถานะ (State Table)

แปลง state diagram ให้เป็นตารางที่เขียนโค้ดต่อได้ทันที

Current State	coin	push	Next State	Output (unlocked)
LOCKED	1	x	UNLOCKED	0
LOCKED	0	x	LOCKED	0
UNLOCKED	x	1	LOCKED	1
UNLOCKED	x	0	UNLOCKED	1

x = don't care (ไม่มีผลต่อผลลัพธ์แถวนั้น)

Verilog (3-Always-Block Pattern)

```
module turnstile_fsm (
    input  clk, rst_n, coin, push,
    output reg unlocked
);
  localparam LOCKED=1'b0, UNLOCKED=1'b1;
  reg state, next_state;
  // 1) State register
  always @(posedge clk or negedge rst_n)
    if (!rst_n) state <= LOCKED;
    else        state <= next_state;
  // 2) Next-state logic
  always @(*)
    case (state)
      LOCKED:   next_state = coin ? UNLOCKED : LOCKED;
      UNLOCKED: next_state = push ? LOCKED   : UNLOCKED;
    endcase
  // 3) Output logic (Moore: f(state) only)
  always @(*) unlocked = (state == UNLOCKED);
endmodule
```

03 · Lab หลัก

Traffic Light Controller

โจทย์: ออกแบบ Moore FSM ควบคุมไฟจราจร 3 สี ให้วนลำดับอัตโนมัติตามเวลา

RED

ติดค้าง 4 วินาที
ก่อนเปลี่ยนเป็น GREEN

GREEN

ติดค้าง 4 วินาที
ก่อนเปลี่ยนเป็น YELLOW

YELLOW

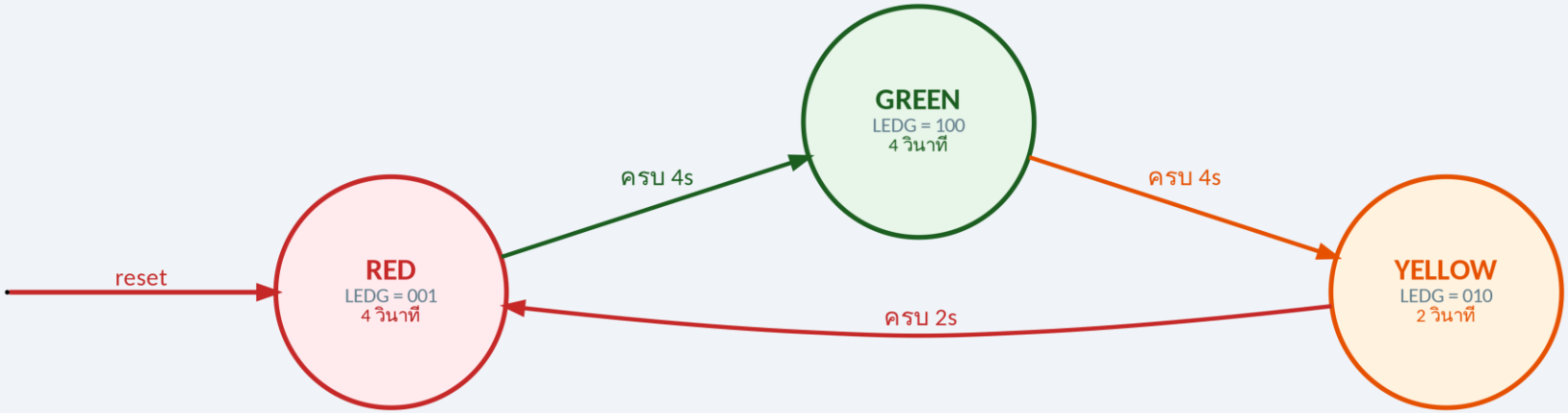
ติดค้าง 2 วินาที
ก่อนวนกลับ RED

I/O บน DE0

CLOCK_50 = clock
KEY[0] = reset (active-low)
SW[0] = enable
LEDG[2:0] = ไฟ 3 สี

State Diagram: RED → GREEN → YELLOW → RED

Moore FSM แบบวงกลม — เปลี่ยนสถานะอัตโนมัติเมื่อครบเวลาที่กำหนด (ไม่ขึ้นกับ input ภายนอกอื่น นอกจาก timer)



รีเซ็ตด้วย KEY[0] → กลับไป RED เสมอ · หยุดนับเวลาเมื่อ SW[0] = 0

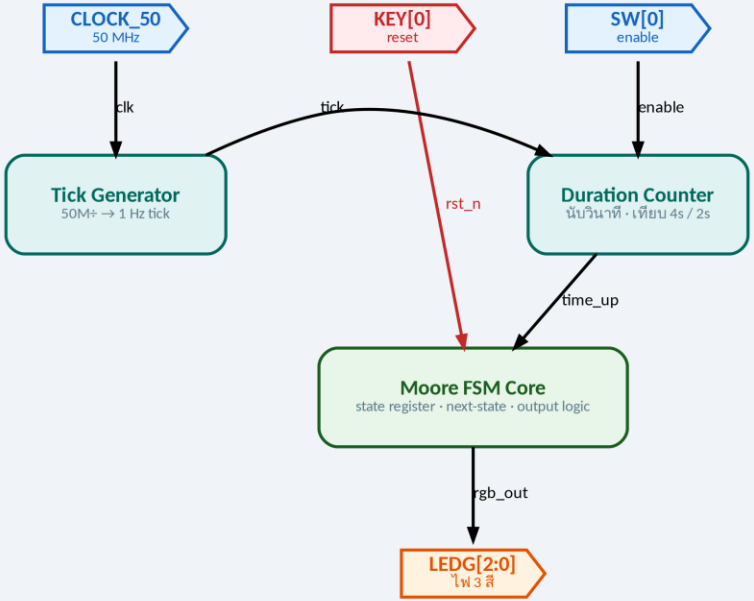
State Encoding + Timing

กำหนดรหัสบิตและระยะเวลาก่อนลงมือเขียนโค้ด

State	รหัส (2 บิต)	ระยะเวลา	LEDG[2:0]	ความหมาย LED
RED	2'b001	4 วินาที	3'b100	LEDG[0] ติด
GREEN	2'b100	4 วินาที	3'b010	LEDG[2] ติด
YELLOW	2'b010	2 วินาที	3'b001	LEDG[1] ติด

โครงสร้างวงจรภายใน

การจับเวลาจริง (วินาที) ต้องหารความถี่ 50 MHz ลงมาก่อน จึงต้องมี 3 บล็อกทำงานร่วมกัน



Verilog Part 1/3 — โมดูลและ Tick Generator

VERILOG · traffic_light_fsm.v (1/3)

```
module traffic_light_fsm (
    input  wire       CLOCK_50,
    input  wire       KEY0,    // active-low reset
    input  wire       SW0,     // enable / run
    output reg  [2:0] LEDG     // {G,Y,R}
);
  localparam RED=2'b00, YELLOW=2'b01, GREEN=2'b10;
  reg [1:0] state, next_state;

  // 1 Hz tick จาก clock 50 MHz
  reg  [25:0] div_cnt;
  wire tick = (div_cnt == 26'd49_999_999);

  always @(posedge CLOCK_50 or negedge KEY0)
    if (!KEY0)     div_cnt <= 0;
    else if (tick) div_cnt <= 0;
    else           div_cnt <= div_cnt + 1;
```

Verilog Part 2/3 — Duration Counter

VERILOG · traffic_light_fsm.v (2/3)

```
reg [2:0] sec_cnt;       // วินาทีที่ผ่านไปใน state นี้
reg [2:0] state_time;    // state นี้ต้องอยู่กี่วินาที

always @(*)
  case (state)
    RED:    state_time = 3'd4;
    YELLOW: state_time = 3'd2;
    GREEN:  state_time = 3'd4;
  endcase

wire last_sec = (sec_cnt == state_time - 1);

always @(posedge CLOCK_50 or negedge KEY0)
  if (!KEY0)              sec_cnt <= 0;
  else if (SW0 && tick)
    sec_cnt <= last_sec ? 0 : sec_cnt + 1;
```

Verilog Part 3/3 — State Register, Next-State, Output

VERILOG · traffic_light_fsm.v (3/3)

```
    // state register
    always @(posedge CLOCK_50 or negedge KEY0)
      if (!KEY0) state <= RED;
      else if (SW0 && tick && last_sec)
        state <= next_state;
    // next-state logic
    always @(*)
      case (state)
        RED:    next_state = GREEN;
        GREEN:  next_state = YELLOW;
        YELLOW: next_state = RED;
      endcase
    // output logic — Moore: f(state) เท่านั้น
    always @(*)
      case (state)
        RED:    LEDG = 3'b001;
        YELLOW: LEDG = 3'b010;
        GREEN:  LEDG = 3'b100;
      endcase
    endmodule
```

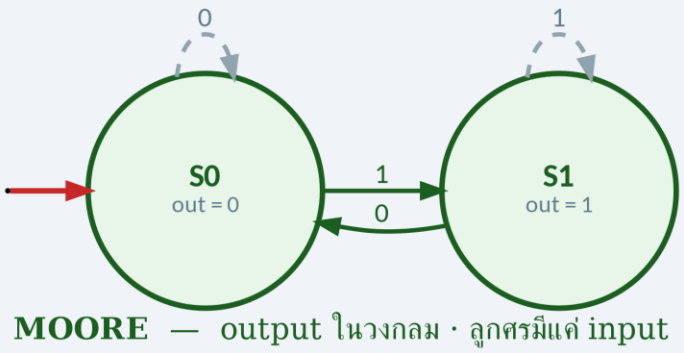
Mealy FSM: Output = f(state, input)

output ผูกกับทั้ง state ปัจจุบันและ input ปัจจุบัน — เปลี่ยนได้แม้ยังไม่ถึง clock edge

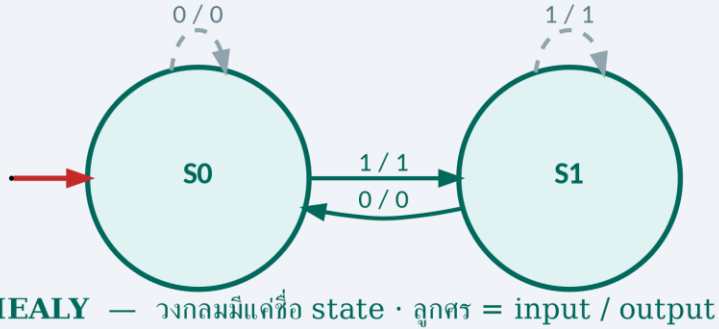
	Moore	Mealy
Output logic	case (state) ... (ไม่มี input)	case (state) if (input) ... (มี input ร่วม)
จำนวน state (pattern เดียวกัน)	มักมากกว่า	มักน้อยกว่า
เสี่ยง glitch จาก input	ไม่มี (output sync กับ clock)	มี (output เปลี่ยนตาม input แบบ async)
เหมาะกับ	ระบบควบคุมที่ output ต้องนิ่ง เช่นไฟจราจร	ระบบตรวจจับ pattern ที่ต้องตอบสนองทันที

สัญกรณ์บน State Diagram: Output เขียนไว้ตรงไหน

ความต่างที่เห็นได้ด้วยตา — Moore เขียน output ไว้ "ในวงกลม state" ส่วน Mealy เขียนไว้ "บนเส้นลูกศร"



MOORE — output ในวงกลม · ลูกศรมีแค่ input

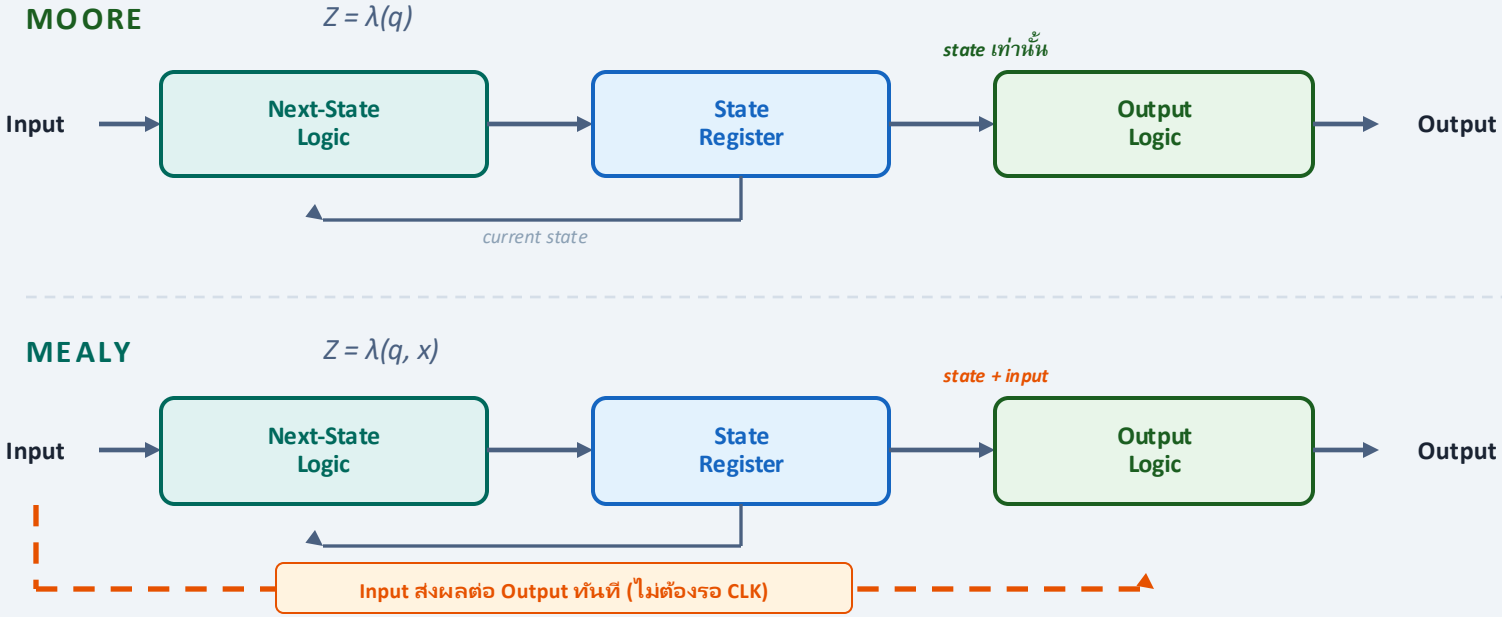


MEALY — วงกลมมีแค่ชื่อ state · ลูกศร = input / output

ตัวอย่างนี้ดัดแปลงจาก GeeksforGeeks "Mealy and Moore Machines in TOC" — self-loop (S0 เจอ 0 คงที่ · S1 เจอ 1 คงที่) วาดครบทุกเส้นแล้วในรูปด้านบน

Block Diagram: สถาปัตยกรรมภายใน Moore vs Mealy

ทั้งสองแบบมี 3 ส่วนเหมือนกัน — ต่างตรงที่ Output Logic ของ Mealy รับ Input โดยตรงด้วย



อ้างอิง: engineeringhulk.com "Finite State Machines — Mealy & Moore"

โครงสร้างโค้ด: 3-Block (Moore) vs 2-Block (Mealy)

Mealy นิยมรวม next-state logic และ output logic เข้าด้วยกัน เพราะทั้งคู่ขึ้นกับ input เหมือนกัน

MOORE · 3 always blocks

```
always @(posedge clk)
  state <= next_state;

always @(*)
  case (state) ...
  next_state = ...

always @(*)  // ไม่มี input!
  case (state) ...
  output = ...
```

MEALY · 2 always blocks

```
always @(posedge clk)
  state <= next_state;

always @(*) begin
  next_state = state;
  output_sig = 0;
  case (state)
    ...  if (input) ...
      output_sig = 1;
  endcase
end
```

Lab B: Sequence Detector หา "1011"

ออกแบบ Mealy FSM ตรวจจับรูปแบบบิต 1011 จากสัญญาณอนุกรม (serial) แบบ overlapping

โจทย์

ตรวจว่าบิตที่ไหลเข้ามาทีละบิต ตรงกับรูปแบบ "1011" หรือไม่

Overlapping

ใช้บิตท้ายของ match หนึ่งเป็นจุดเริ่มของ match ถัดไปได้ เช่น 1011011 เจอ 2 ครั้ง

Output

detected = 1 ทันที 1 step ที่ครบรูปแบบ (นี่คือจุดเด่นของ Mealy)

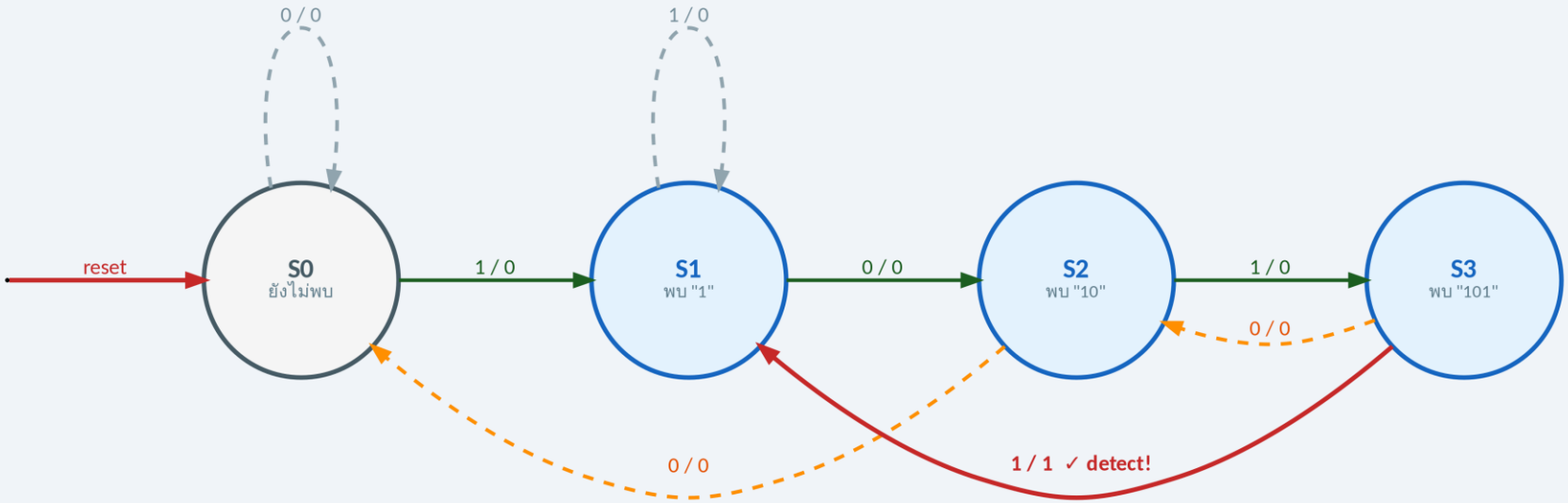
I/O บน DE0

KEY[0]=reset
KEY[1]=step (นาฬิกากดทีละครั้ง)
SW[0]=data bit
LEDG[0]=detected
HEX0=จำนวนครั้ง

ตัวอย่าง: ป้อนบิต 1-0-1-1-0-1-1 ตามลำดับ → เจอ "1011" ที่บิตที่ 4 (บิต 1-4) และซ้ำอีกครั้งที่บิตที่ 7 (บิต 4-7) — บิตที่ 4 ถูกใช้ร่วมกันในทั้งสอง match นี่คือ overlapping

State Diagram: Sequence Detector "1011"

ลูกศรบน state diagram ของ Mealy เขียนกำกับด้วย "input / output" — สีเขียว=เดินหน้าตรงรูปแบบ, สีแดง=ตรวจพบครบ, สีเทา=พบ 0 (บางส่วน)



ตารางสถานะ (State Table) — ครบทุก Transition

ใช้ตารางนี้เป็นข้อมูลอ้างอิงตอนเขียนโค้ด case ให้ครบทุกกรณี

Current State	Input	Next State	Output (detected)
S0	0	S0	0
S0	1	S1	0
S1	0	S2	0
S1	1	S1	0
S2	0	S0	0
S2	1	S3	0
S3	0	S2	0
S3	1	S1	1

Verilog Part 1/2 — โมดูลและ State Register

VERILOG · seq_detector_1011.v (1/2)

```
module seq_detector_1011 (
    input  wire clk,      // KEY[1]
    input  wire rst_n,    // KEY[0] — reset
    input  wire in,       // SW[0] — data bit
    output reg  detected // LEDG[0]
);
  localparam S0=2'b00, S1=2'b01, S2=2'b10, S3=2'b11;
  reg [1:0] state, next_state;

  // state register
  always @(posedge clk or negedge rst_n)
    if (!rst_n) state <= S0;
    else        state <= next_state;
```

Verilog Part 2/2 — Next-State + Output (Mealy)

VERILOG · seq_detector_1011.v (2/2)

```
  // next-state + output logic รวมกัน (Mealy: f(state,input))
  always @(*) begin
    next_state = state;  // กัน latch
    detected   = 1'b0;      // กัน latch
    case (state)
      S0: next_state = in ? S1 : S0;
      S1: next_state = in ? S1 : S2;
      S2: next_state = in ? S3 : S0;
      S3: if (in) begin
            next_state = S1;
            detected   = 1'b1; // ครบ "1011"!
          end else begin
            next_state = S2;  // เก็บ "10" ไว้
          end
    endcase
  end
endmodule
```

5 ตัวอย่าง FSM ในงานจริง

เรียงตามระดับความซับซ้อน — ตั้งแต่ 2 สถานะ จนถึง 6 สถานะ

01 ★☆☆☆☆

Edge Detector
ตัวตรวจขอบ

Mealy FSM
2 สถานะ

ตรวจจับ rising edge
จาก 0→1 ส่ง pulse
หนึ่งรอบสัญญาณนาฬิกา

02 ★★☆☆☆

Fan Controller
พัดลมอัตโนมัติ

Moore FSM
3 สถานะ

ควบคุมพัดลมตาม
อุณหภูมิพร้อม
hysteresis

03 ★★★☆☆

Vending Machine
ตู้ขายน้ำ

Mealy FSM
4 สถานะ

รับเหรียญ 5¢/10¢
จ่ายสินค้า + ทอน
เมื่อครบ 15¢

04 ★★★★☆

UART Receiver
รับข้อมูลอนุกรม

Moore FSM
4 สถานะ + counter

รับ serial data
start → 8 bit → stop
ตรวจ frame error

05 ★★★★★

Washing Machine
เครื่องซักผ้า

Moore FSM
6 สถานะ

วงจรซัก 6 ขั้นตอน
หลาย I/O
lid_open safety

LEVEL 1 · BASIC

ตัวอย่าง 1 — Edge Detector (ตัวตรวจขอบสัญญาณ)

Mealy FSM · 2 สถานะ · ตรวจจับ rising edge (0→1) แล้วส่ง pulse ออก 1 clock cycle

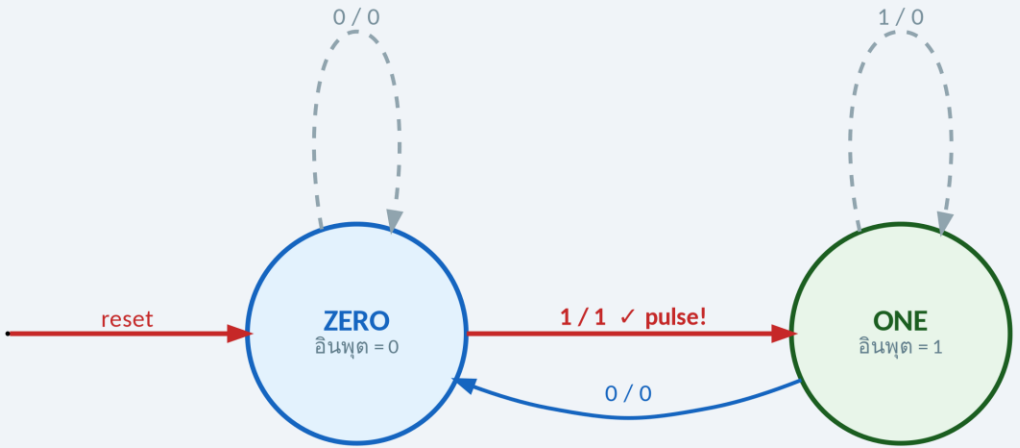
ชนิด FSM
Mealy — output ขึ้นกับ state + input

จำนวนสถานะ
2 สถานะ: ZERO, ONE

อินพุต
d_in — สัญญาณดิจิทัลที่ต้องการตรวจ

เอาต์พุต
pulse — ส่ง 1 เฉพาะเมื่อ 0→1 (1 clk)

การใช้งาน
ตรวจขอบปุ่มกด, สร้าง single-pulse จาก slow signal, debounce



💡 **Mealy ดีกว่า Moore ตรงนี้เพราะ:** output เปลี่ยนทันทีที่ input เปลี่ยน → pulse เร็วกว่า 1 clk

★☆☆☆☆

LEVEL 2 · INTERMEDIATE

ตัวอย่าง 2 — Automatic Fan Controller (พัดลมอัตโนมัติ)

Moore FSM · 3 สถานะ · ควบคุมความเร็วพัดลมตามอุณหภูมิพร้อม hysteresis

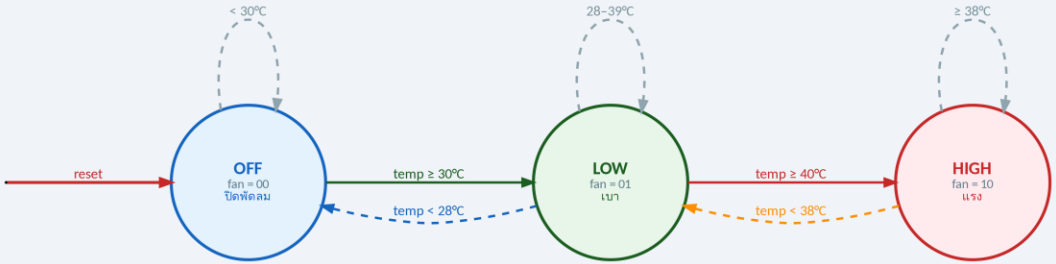
ชนิด FSM
Moore — output ขึ้นกับ state เท่านั้น

จำนวนสถานะ
3 สถานะ: OFF, LOW, HIGH

อินพุต
temp[7:0] — ค่าอุณหภูมิจาก sensor

เอาต์พุต
fan[1:0] — 00=ปิด, 01=เบา, 10=แรง

จุดเด่น
Hysteresis — เปิดที่ 30°/40°C
ปิดที่ 28°/38°C ป้องกัน oscillation



💡 **Hysteresis:** เกณฑ์เปิด ≠ เกณฑ์ปิด เพื่อป้องกันพัดลมสั่นเปิด-ปิดรัวเมื่ออุณหภูมิอยู่ใกล้จุดตัด

★★☆☆☆

LEVEL 3 · APPLIED

ตัวอย่าง 3 — Vending Machine (ตู้ขายน้ำอัตโนมัติ)

Mealy FSM · 4 สถานะ · รับเหรียญ 5¢ / 10¢ · จ่ายสินค้าเมื่อครบ 15¢ · ทอนเงิน

ชนิด FSM

Mealy — output (ทอน/จ่าย) ขึ้นกับ state + เหรียญที่หยอด

จำนวนสถานะ

4 สถานะ: IDLE, S5, S10, S15

อินพุต

coin[1:0] — 00=ไม่หยอด, 01=5¢, 10=10¢

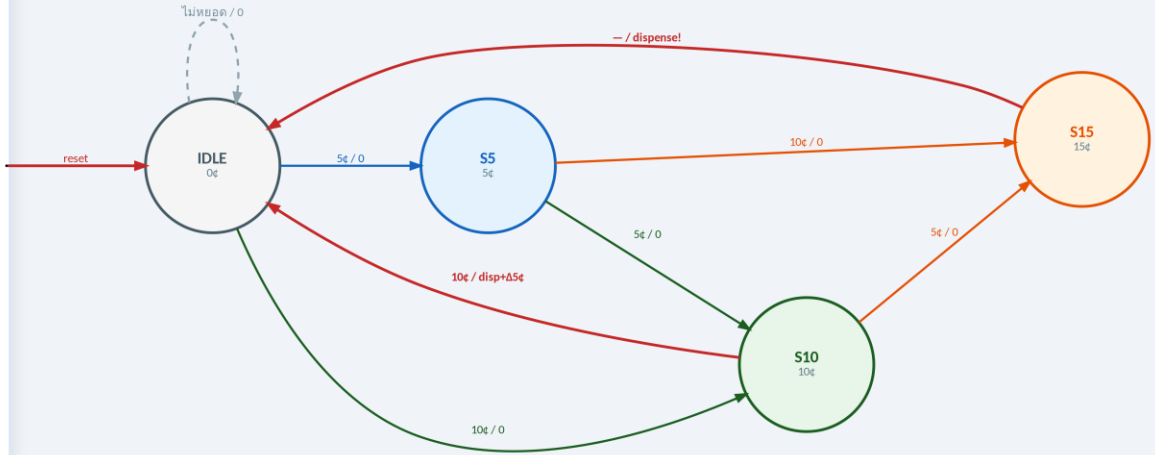
เอาต์พุต

dispense, change — จ่ายสินค้า + ทอน

จุดเด่น

Overpay: หยอด 10¢ ตอนมี 10¢

→ จ่ายสินค้า + ทอน 5¢



★★★☆☆

LEVEL 4 · ADVANCED

ตัวอย่าง 4 — UART Receiver (ตัวรับข้อมูลอนุกรม)

Moore FSM · 4 สถานะ + bit counter · รับ serial data 8N1 · ตรวจ frame error

ชนิด FSM

Moore — output ขึ้นกับ state เท่านั้น

จำนวนสถานะ

4 สถานะ: IDLE, START, DATA, STOP

+ bit_cnt[2:0] counter

อินพุต

rx — สาย serial data (idle = 1)

เอาต์พุต

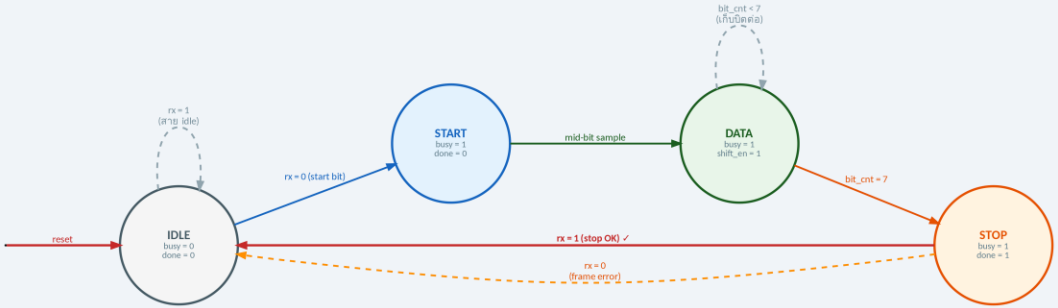
busy, done, shift_en, data_out[7:0]

จุดเด่น

FSM + datapath: ใช้ counter นับ 8 บิต

+ shift register เก็บ data

+ frame error detection



★★★★☆

LEVEL 5 · COMPLEX

ตัวอย่าง 5 — Washing Machine Controller (เครื่องซักผ้า)

Moore FSM · 6 สถานะ · หลาย I/O · safety interlock (lid_open)

ชนิด FSM

Moore — output = f(state)

จำนวนสถานะ

6: IDLE ⟶ FILL ⟶ WASH ⟶ DRAIN ⟶ SPIN ⟶ DONE

อินพุต

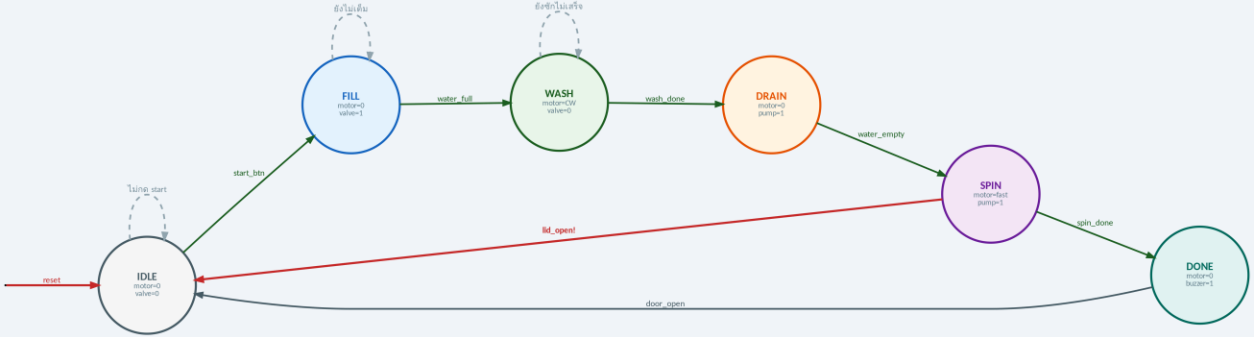
start_btn, water_full, wash_done, water_empty, spin_done, lid_open, door_open

เอาต์พุต

motor[1:0], valve, pump, buzzer

จุดเด่น

Safety: lid_open ระหว่าง SPIN ⟶ หยุดทันที (emergency stop) ป้องกันอันตราย



★★★★★

COMPARISON

เปรียบเทียบ 5 ตัวอย่าง FSM

สรุปจุดสำคัญแต่ละตัวอย่าง — จำนวนสถานะ, ชนิด FSM, I/O, ความซับซ้อน

ตัวอย่าง	สถานะ	ชนิด	อินพุต / เอาต์พุต	จุดเรียนรู้
1. Edge Detector	2	Mealy	d_in → pulse	FSM 2 สถานะ เรียบง่ายที่สุด output = f(state, input)
2. Fan Controller	3	Moore	temp → fan[1:0]	Hysteresis ป้องกัน oscillation output = f(state)
3. Vending Machine	4	Mealy	coin → disp, change	Multiple transitions จาก 1 state overpay handling
4. UART Receiver	4+cnt	Moore	rx → busy, done, data	FSM + datapath (counter, shift reg) frame error detection
5. Washing Machine	6	Moore	7 inputs → 5 outputs	Safety interlock (lid_open) complex sequencing

DESIGN TIPS

แนวทางออกแบบ FSM ในงานจริง

เทคนิคที่เห็นจากตัวอย่าง — นำไปใช้ได้กับ FSM ทุกขนาด

1. เลือก Moore vs Mealy ให้เหมาะ — ถ้า output ต้องตอบสนองทันที (เช่น pulse) → Mealy · ถ้า output ต้องเสถียร → Moore
2. ใช้ Hysteresis เมื่อ input มี noise — กำหนดเกณฑ์เปิด ≠ เกณฑ์ปิด เพื่อป้องกัน state สั่น (oscillation)
3. แยก FSM กับ Datapath — ถ้ามี counter / shift register → ให้ FSM สั่ง enable แล้วแยก counter เป็นโมดูลข้างนอก
4. คิดถึง Safety เสมอ — ออกแบบ transition สำหรับกรณีอันตราย (เช่น lid_open → emergency stop) ก่อนเรื่องอื่น
5. วาด State Diagram ก่อนเขียนโค้ด — จะเห็น transition ที่ขาดหาย, dead state, หรือ unreachable state ได้ง่าย

สรุปสัปดาห์ที่ 7-8

- FSM = state + input + output + transition rule ✓
- Moore: output = f(state) เท่านั้น · Mealy: output = f(state, input) ✓
- Lab A — Traffic Light Controller (Moore, 3-always-block) บน DE0 ✓
- Lab B — Sequence Detector "1011" (Mealy, overlapping) บน DE0 ✓
- เข้าใจ trade-off: จำนวน state, ความเสี่ยง glitch, และการเลือกใช้ Moore vs Mealy ✓
- เห็นตัวอย่างจริง: Edge Detector, Fan, Vending Machine, UART, Washing Machine ✓

▶ สัปดาห์ที่ 9: Testbench Design & Functional Verification — Self-Checking Testbench สำหรับ FSM และ Counter จากสัปดาห์ก่อนหน้า